

SPECTRAL A-N V18: SOFTWARE ARCHITECTURE AND REPRODUCIBILITY COMPANION

ELIUTH CHAVERO JASSO

ABSTRACT. This companion documents the software architecture, artifact schemas, hashing/provenance contracts, and reproducibility discipline of the v18 spectral certification suite (`spectral/certification_v18/`, 85 passing tests as of this writing). It records a real, reproducible CPU/GPU parity result on the confirmed hardware (RTX PRO 5000 Blackwell, 24GB VRAM), a concrete process-global TF32 contamination hazard found and fixed, and the current gaps in continuous integration and packaging for this new track.

1. REPOSITORY LAYOUT

Path	Contents
<code>spectral/legacy/v17/</code>	Immutable, hash-verified legacy script + run-log copies
<code>spectral/certification_v18/gates.py</code>	Typed-state enum, fail-closed combination logic
<code>spectral/certification_v18/config.py</code>	Certification-mode contract, enforced via <code>validate()</code>
<code>spectral/certification_v18/model.py</code>	Fresh CP-law/projector geometry (not imported from legacy)
<code>spectral/certification_v18/blocks/</code>	One module + one <code>BLOCK*_FINDINGS.md</code> per block
<code>spectral/certification_v18/hardware/</code>	Certification-mode setup, resumable job queue
<code>spectral/certification_v18/artifacts/</code>	Saved experiment result JSON
<code>spectral/certification_v18/tests/</code>	85 tests, <code>pytest spectral/certification_v18/tests</code>

This track is deliberately NOT reimplemented into `src/seion_core`: it is scope class `SPECTRAL_LEGACY_TRACK` (`claims/scope_registry_v4.yaml`), evidentially separate from `CANONICAL_FINITE_CORE` by design.

2. WHY `MODEL.PY` IS A FRESH REIMPLEMENTATION, NOT AN IMPORT

The legacy script sets `torch.backends.cuda.matmul.allow_tf32 = True` and `torch.set_float32_matmul_precision("high")` unconditionally at import time when CUDA is available. These are process-global flags, not scoped to the importing module. A certification-mode process that imported the legacy script for convenience would have its TF32 discipline silently overridden by an unrelated import. ‘`model.py`’ reimplements the needed primitives (CP product, projector, associator, reduced curvature) independently, with its own tests (`test_model.py`) checked against the legacy formulas by direct line-number citation, and `hardware/certification_mode.py` provides `enter_certification_mode()` / `assert_certification_discipline()` as the single, idempotent, re-assertable enforcement point.

Date: 30 July 2026.

Companion to “A Fail-Closed Certification Framework for Cyclic N-Ary Laws, Projectors, and Multiresolution Tensor Structure” (`papers/a_to_n_certification_v18`). Documents interfaces and evidence contracts; makes no independent mathematical claim.

3. TYPED-GATE SCHEMA

`gates.py` defines `TypedStatus` (10 states) and `CRITICAL_GATES` (8 gates, each a tuple of contributing block names). `combine_gate_status` takes the minimum over a gate’s blocks (excluding `NOT_APPLICABLE`, which is tracked separately and never silently counted as passing). `assign_block_status` is the single enforcement point preventing a screening-mode run from being assigned a certificate-tier status: it raises `ScreeningCertificateViolation` rather than downgrading silently.

4. CPU/GPU PARITY: A REAL, EXECUTED RESULT

On the confirmed hardware (Intel Core Ultra 9 285HX, 24 logical cores, 128 GB RAM; NVIDIA RTX PRO 5000 Blackwell Laptop GPU, 24 GB VRAM, CUDA 12.8, driver 573.49), we constructed a `SpectralModelV18` instance on CPU (float64, seed 0), transferred its exact parameter/buffer values to an independently-constructed CUDA instance via `state_dict` (not `.to(device)`, which does not update the plain Python `device/dtype` string attributes stored on `CyclicCPPProduct` — a second real hazard found during this exercise, documented in `model.py`), and computed Block A/B quantities on both devices with TF32 explicitly disabled and deterministic algorithms enabled. Results:

Quantity	CPU	CUDA
idempotence residual	1.312×10^{-15}	1.198×10^{-15}
self-adjointness residual	0.0 (exact)	2.220×10^{-16}
comm. unexplained-rel (random init)	1.0000380759060201	1.0000380759060197
wall time	19.1 ms	835.1 ms

The unexplained-rel figure (the only quantity here large enough for a relative-difference comparison to be meaningful) agrees to 4.4×10^{-16} relative difference — CPU/GPU parity confirmed at float64 for this computation. The wall-time comparison is an honest negative result for accelerating problems at this scale: GPU kernel-launch overhead dominates for $n = 24$, and CPU is $\sim 44\times$ faster in this single case. Real GPU throughput advantage should be expected only once batch sizes or ambient dimensions are large enough to amortize that overhead — not exercised in this pass; the sweep/scaling phase (Section 7) is where that would be measured.

5. RESUMABLE JOB QUEUE

`hardware/job_queue.py` implements an append-only, JSONL-backed ledger keyed by `scientific_instance_id` (the mathematical configuration) versus `execution_id` (one specific attempt), recording seed, precision, hardware, a config hash, a script hash, checkpoint lineage (parent `execution_id`), status, retry count, and output hashes. History is never mutated, only appended to, matching the pattern already established for the separate `CANONICAL_FINITE_CORE` track’s `src/seion_core/governance/runs.py` (different schema, same non-destructive philosophy, since this track’s artifact set does not match that track’s `REQUIRED_RUN_ARTIFACTS` contract). Tested: submit/complete, retry-increments-count while preserving prior history, and checkpoint-lineage traversal.

6. LEGACY PROVENANCE

`spectral/certification_v18/tools/ingest_legacy.py` hashes both named legacy files (sha256, verified after copy), hashes all 18 on-disk run directories, parses the 9-run text log, cross-references it against on-disk `summary.json` files, reconstructs resume lineage, and reclassifies every historical run against the v18 typed-gate vocabulary. Output: `spectral/legacy/v17/legacy_a_to_n_manifest.yaml`, `legacy_run_lineage.json`, `legacy_run_dedup_report.md`, `legacy_claim_reclassification.yaml`.

7. GAPS AND FOLLOW-UP (HONESTLY LISTED, NOT HIDDEN)

- No CI workflow for `spectral/certification_v18/` exists yet in `.github/`; the 85 tests are currently run manually only.
- No SBOM or packaging metadata specific to this track has been generated; the track depends on the same `pyproject.toml` dependencies (`torch`, `numpy`) as the rest of the repository, declared under the existing `research` extra.
- GPU throughput scaling (large batch/dimension sweeps) was not measured — only correctness parity at small scale.
- Clean-environment reproduction (a fresh `virtualenv`/container install-and-rerun) was not exercised this pass; all runs used the already-configured local environment.

INDEPENDENT RESEARCHER, APIZACO, TLAXCALA, MEXICO